Cloud & AWS Storage

By/menna samy

1. From an online store to a cloud giant

Amazon started out as nothing more than an e-commerce company — and ended up becoming one of the giants of cloud computing. So how did one lead to the other?



Amazon: from a simple e-commerce company to one of the giants of cloud computing

As Amazon's business started to expand, dealing with the infrastructure became very tiring and very costly.

So they started thinking: why don't we have a set of **reusable services**, so we no longer have to start from scratch every single time? That's exactly what happened — and it didn't stay inside Amazon: it became available to other companies as a service. And from the very beginning, the first of these services were **EC2** and **S3**.

> *~ the infrastructure Amazon built to solve its own pain turned into a service that everyone else could use too.*

---

2. Cloud + Docker: deployment for everyone

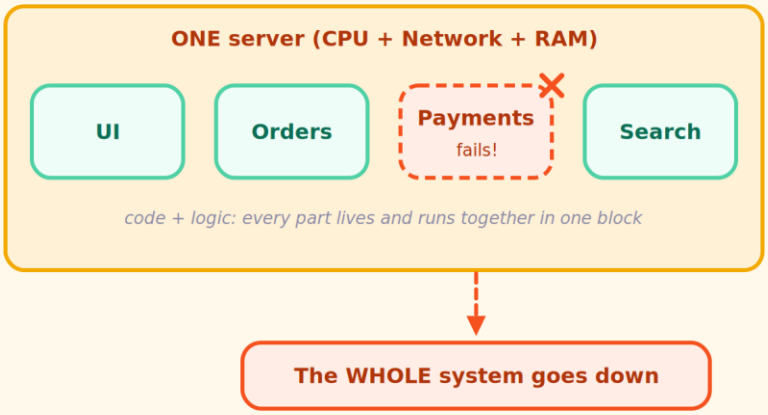
Before microservices became widespread, the cloud already existed. But having the cloud together with Docker is what made it possible to deploy your application much more easily — and made that available to everyone.

3. Before microservices: the monolith

Before microservices, everything was **monolithic**: all of the code, and all the parts of the application that interact with each other, ran together on one single server.

Problem 1: one part falls, everything falls

Since everything lived and ran together, if just one part of it went down, the whole system went down with it.



Problem 2: scaling means scaling everything

When it was time to scale, the options were: raise the server's capabilities (more CPU, network and RAM), or — even with horizontal scaling — take the entire system and copy it several times, even when the load was concentrated on only one part of it.

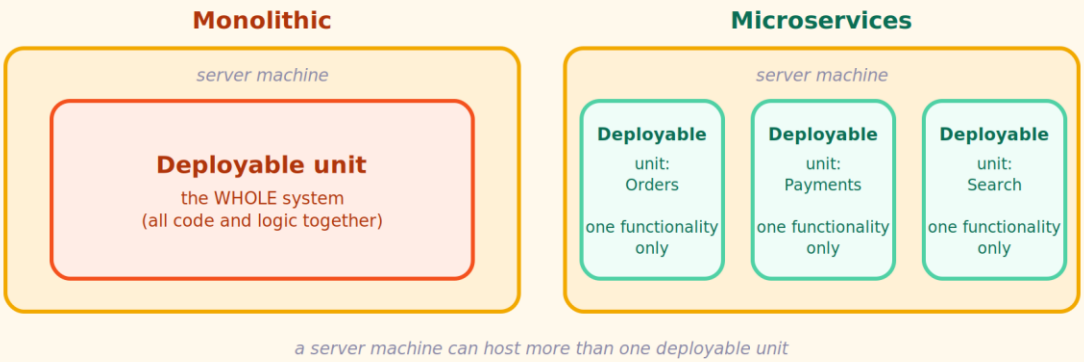


The result? **Waste of resources.** You end up paying for full copies of the whole system just to serve the pressure on a single part.

4. Two meanings of the word "server"

This is where an important distinction comes in. The commonly used term *server* is often used to mean your **app code** — but a *server* can also mean a **physical machine**. Two different things, one shared name.

The more accurate term for the thing that carries your code is **deployable unit**. A server machine can even host more than one deployable unit.



a server machine can host more than one deployable unit

- **Monolithic** → the deployable unit is the whole system.
- **Microservice** → the deployable unit is one part, responsible for one functionality only.

5. Why microservices help

Since each part is its own deployable unit:

- If one part fails, the system keeps working.
- I can do scaling for specific parts only — the ones that actually need it.
- Each part can use the tech stack that suits it best.

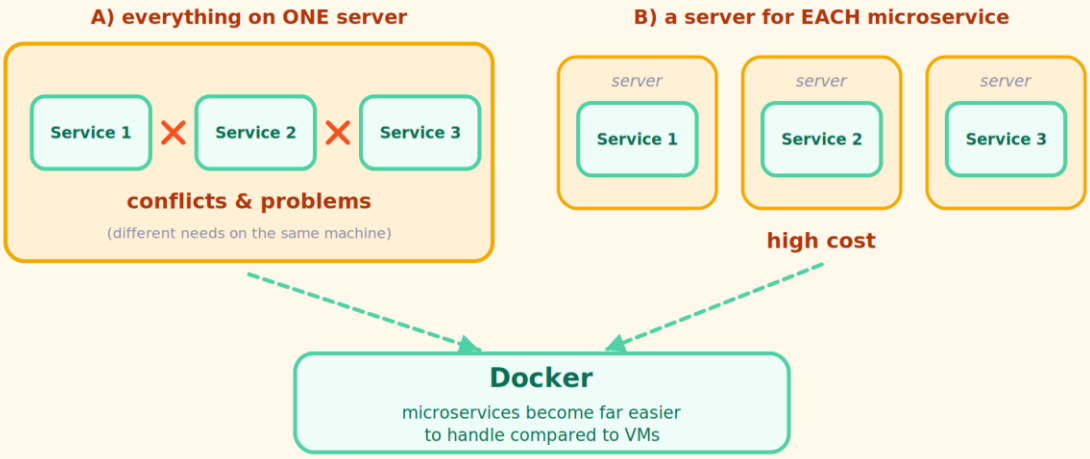


6. The catch: infrastructure for every microservice

Now imagine that every single microservice needs its own infrastructure to be prepared for it. So the idea is lovely — but costly, especially before Docker existed.

The dilemma before Docker

- Put all the microservices on the **same server** → conflicts and problems between them.
- Give each microservice **its own server** → high cost.



Then Docker showed up, and working with microservices became far easier compared to VMs.

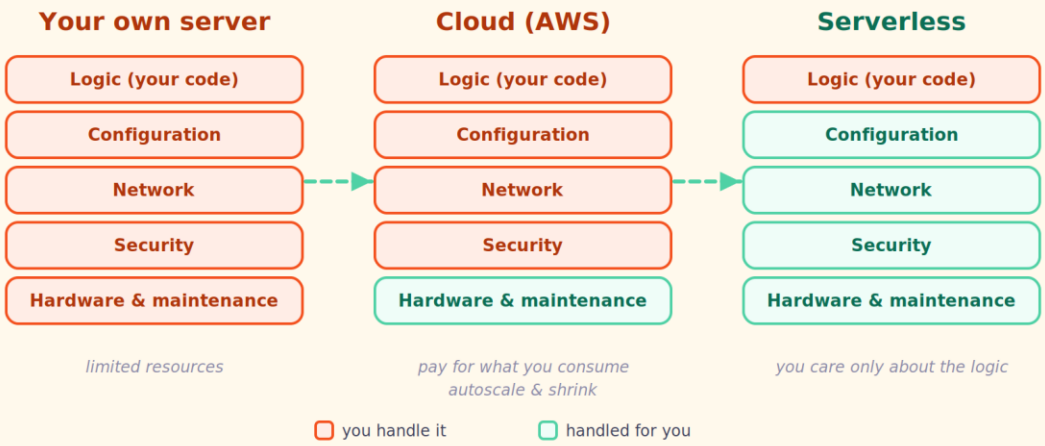
7. AWS made it even easier

The existence of AWS made things easier and easier. Today you're no longer required to set up a real server and look after it and maintain it — and a physical server had limited resources to begin with.

Instead, you take just the resources you need, as much as you actually consume, and you can **autoscale** up and **shrink** back down whenever it is needed.

And that's not all. With the arrival of **serverless**, you stopped worrying about the server's problems altogether — its configuration, its network and its security — and cared only about the **logic**.

Public


Your own server
Logic (your code)
Configuration
Network
Security
Hardware & maintenance
limited resources
Cloud (AWS)
Logic (your code)
Configuration
Network
Security
Hardware & maintenance
pay for what you consume
autoscale & shrink
Serverless
Logic (your code)
Configuration
Network
Security
Hardware & maintenance
you care only about the logic
you handle it
handled for you

8. Floci + AWS CLI

The idea is to work with AWS storage services **locally on your own machine**, without needing a real AWS account.

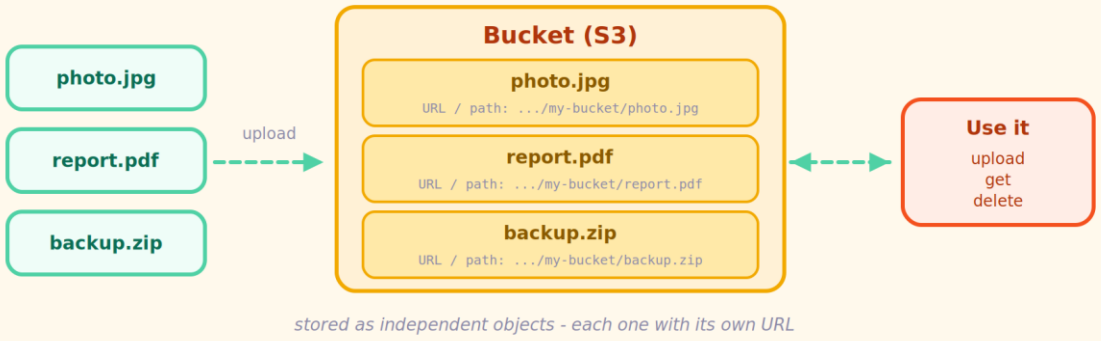
Role of Floci: a local runtime that simulates the AWS service interfaces you need for practice. The **AWS CLI** is the tool you use to send the commands.

9. Understanding the storage types first

Before touching any commands, let's make sure the kinds of storage involved are clear.

Object Storage (like S3)

- **What does it store?** Raw files: images, videos, PDFs, audio files and backups.
- **How does it store?** Data is kept as "packages" or independent files, and every file gets its own link or unique path (URL).
- **Can we search inside it?** Hard and slow. It isn't designed for you to go inside the files and search, or to edit one specific line in a text file. It's a system of: *"upload the file, fetch the file, delete the file."*



Block Storage (like EBS)

- **What does it store?** Parts of a virtual hard disk, attached to a server (EC2) so that it can run on it.
- **How does it store?** The storage is divided into blocks, and the server sees this disk as if it were a real hard drive plugged into its motherboard — so it can install an operating system (OS) on it or run programs.
- **The difference from S3:** EBS needs a server to control it (like a computer's disk), while S3 is an independent service reachable over the internet without a direct server.

Regular databases (like MySQL or MongoDB)

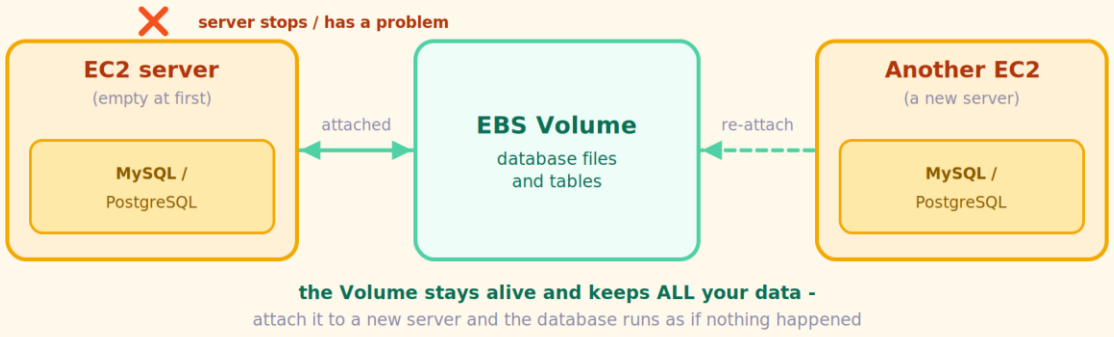
- **What do they store?** Structured data: usernames, passwords, product details and payment records.
- **How do they store?** In tables or in related documents.
- **Why don't we use S3 for databases?** Because a database is designed for very fast search, instant modification of a tiny part of the data (Query/Update), and guaranteeing data integrity (Transactions) — and all of that is impossible or very slow on object storage like S3.

How does a database run on EBS?

1. Suppose we created a virtual server on AWS (EC2). At first, this server is empty.
2. To install a database program on it (such as MySQL, PostgreSQL or MongoDB), we need a hard disk to store the database's files and tables — which keep growing and shrinking all the time.
3. So we attach the EC2 server to an **EBS Volume**, and let the database store its files inside it.

Benefits of storing a database on EBS:

- **Safety and persistence of data:** if the server (EC2) stops or runs into a problem and gets wiped, the EBS Volume stays there, holding all of your database's data — and you can attach it to another server, and the database runs as if nothing happened.
- **High performance:** EBS is designed to give very high read/write (I/O) speeds — which is vital for databases, because they perform reads and writes all the time.



10. Preparing the tools

You'll need three things:

- **Docker Desktop** — to run the containers.
- **AWS CLI** — to send AWS commands from the Terminal.
- **Floci** — the emulator the section asks for.

Open a Terminal or PowerShell and test the tools:

```
docker --version
aws --version
```

If a version number shows up for each tool, it means they're installed and can be called from the command line. Also make sure Docker Desktop is actually running.

11. Running Floci on your machine

There are two ways. For the simplest one, start with the first — it fits this section's content well. **Pick one way only; you don't need to run both.**

Way 1: with Docker

```
docker run -d --name floci-emulator -p 4566:4566 floci/floci:latest
```

- `docker run` — starts a new container.
- `-d` — runs it in the background.
- `--name floci-emulator` — gives it a name that's easy to work with.
- `-p 4566:4566` — connects your machine's port to Floci's port inside the container.
- `floci/floci:latest` — uses the official Floci image.

Then make sure the container is running — you should find `floci-emulator` among the running containers — and, if you want, look at its startup log:

```
docker ps
```

Port **4566** is the local address you'll point the AWS CLI at.

Way 2: with the Floci CLI

Floci also provides its own tool that starts the emulator and sets up the connection variables automatically — check the official Floci installation guide. Once it's installed:

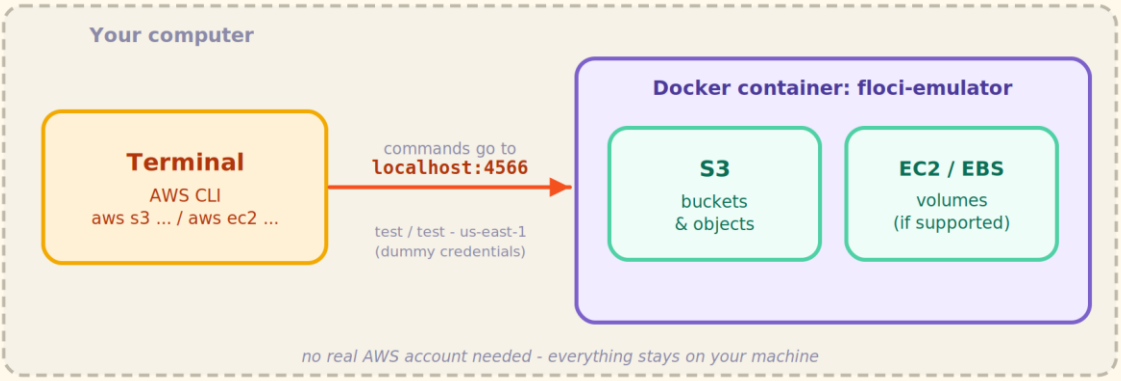
```
floci start
```

- Linux, Git Bash or macOS: `eval $(floci env)`
- PowerShell: `floci env | Invoke-Expression`

12. Pointing the AWS CLI at Floci

This is one of the most important steps in the section. Normally, AWS CLI commands connect to the **real** AWS services. We want them to connect to the local emulator instead.

We do that with **environment variables**, which set Floci's address and the dummy login details.



Variable	What it does
AWS_ENDPOINT_URL	Points the AWS CLI commands to Floci
AWS_ACCESS_KEY_ID	A dummy access key
AWS_SECRET_ACCESS_KEY	A dummy secret key
AWS_DEFAULT_REGION	The default region

Windows PowerShell

Run these in the same PowerShell window:

```
$env:AWS_ENDPOINT_URL="http://localhost:4566"
$env:AWS_ACCESS_KEY_ID="test"
$env:AWS_SECRET_ACCESS_KEY="test"
$env:AWS_DEFAULT_REGION="us-east-1"
```

Git Bash, Linux or macOS

```
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
```

13. S3 Bucket — Object Storage

Step 1: create a Bucket

```
aws s3 mb s3://my-local-bucket
```

- `aws s3` — work with the S3 service.
- `mb` — short for Make Bucket, i.e. create a Bucket.
- `my-local-bucket` — the name of the Bucket you're creating.

You should see a response similar to `make_bucket: my-local-bucket`. If the name is already taken, choose a different one.

Step 2: create a sample file

```
echo "Hello from Floci!" > sample.txt
```

This creates a text file named `sample.txt` and writes a message inside it.

Step 3: upload the file to S3

```
aws s3 cp sample.txt s3://my-local-bucket/
```

`cp` means Copy — in this example it uploads the file from your machine to the Bucket.

Step 4: make sure it worked

```
aws s3 ls s3://my-local-bucket/
```

You should find `sample.txt` among the files listed.

That completes the first part: you created Object Storage and uploaded an Object into it.

~ these commands match Floci's official getting-started guide.

14. EBS-equivalent Volume — Block Storage

Now the second part. In AWS, EBS provides a **Volume** that works as a storage disk for EC2.

> **Watch out:** creating a Volume does not mean we created an EC2 Instance or attached the disk to a server.

Step 1: create a Volume

```
aws ec2 create-volume --size 10 --availability-zone us-east-1a --volume-type gp2
```

Option	Meaning
ec2	The EC2 service
create-volume	Create a Volume
--size 10	A size of 10 GiB
--availability-zone us-east-1a	Specifies the availability zone
--volume-type gp2	The type of the Volume

If the operation is supported in your Floci version, you should get back a response containing the Volume's details — including its **VolumeId** and **State**.

Step 2: list the Volumes

```
aws ec2 describe-volumes
```

Look in the output for the Volume you created, and check its size, its ID and its state.